

Weekly Interview Preparation

DSA + Git & GitHub + Full-Stack Technical Interview + Frontend Machine Coding

● DSA — LeetCode

● Git & GitHub

● Full-Stack Technical

● Machine Coding

Objective — Practice how to solve, explain, build, communicate and document. Not how to memorize answers.

SECTION	WEEKLY WORK	EVIDENCE
1 DSA (LeetCode)	5 problems	Accepted screenshots + Drive folder
2 Git & GitHub	5 scenario-based questions	Recorded answers for selected questions
3 Full-Stack Technical	5 interview questions	Recorded answers for selected questions
4 Machine Coding	1 frontend task	GitHub repo + live link + demo

1 DSA (LeetCode)

SECTION 1

Solve all 5 directly on LeetCode. Do not copy a solution before attempting the problem yourself. After an Accepted submission, upload proof to your weekly Drive folder.

#	PROBLEM	LEVEL	LEETCODE
01	Two Sum	Easy	LC 1
02	Remove Duplicates from Sorted Array	Easy	LC 26
03	Move Zeroes	Easy	LC 283
04	Rotate Array	Medium	LC 189
05	3Sum	Medium	LC 15

DSA submission: Upload one screenshot per accepted problem showing the problem/solution result and your accepted status. Put all 5 screenshots in the Sheet 01 → DSA Drive folder and submit that folder link.

Answer these like an interview. First explain what you would do, then explain why. Be ready for the follow-ups.

1 Wrong Branch, Correct Work

SCENARIO You accidentally made 4 commits on main instead of your feature branch. How would you move the work to the correct branch without losing it?

- Would your approach change if the commits were already pushed to the remote?
- When would you use cherry-pick here?
- What would you do if teammates were already using main?

2 Merge Conflict During PR

SCENARIO Your feature branch has a merge conflict with main just before the PR is merged. Walk through the complete process of resolving it safely.

- How do you verify that the conflict resolution did not break existing functionality?
- Would you merge main into the feature branch or rebase? Explain the trade-off.
- What should you do before pushing the resolved branch?

3 Secret Accidentally Committed

SCENARIO A developer committed an API key to GitHub and immediately realizes the mistake. What steps should they take?

- Why is deleting the commit/file not enough?
- What should happen to the exposed key itself?
- How would you prevent similar secrets from being committed again?

4 PR Review With New Changes

SCENARIO A reviewer asks for changes on your pull request after you have already pushed several commits. How would you update the PR while keeping the review understandable?

- Would you create a new commit or amend/squash?
- When can force-push become risky?
- How would you communicate a significant change to the reviewer?

5 Recovering Lost Local Work

SCENARIO You ran a Git command incorrectly and your local branch no longer points to the commit containing yesterday's work. How would you investigate and recover it?

- What is git reflog and why can it help?
- How would you verify the recovered commit before changing the branch?
- What habit would reduce the chance of permanent loss?

This section is cumulative. Questions can come from JavaScript, React, Node.js, Express, REST APIs, MongoDB/Mongoose, authentication/authorization, ImageKit and — once taught — TypeScript, Next.js and other full-stack topics. This week's questions stay within topics already taught, while the bank expands as the course progresses.

1 JavaScript — Async Flow

QUESTION You call an API using `fetch()`, update the UI after the response, and also have a `setTimeout()` in the same flow. Explain how JavaScript's event loop affects the order of execution.

- What is the difference between a microtask and a macrotask?
- Where do Promise callbacks fit?
- Why can blocking synchronous JavaScript make the UI feel frozen?

2 React — Unnecessary Re-render

QUESTION A React page becomes slow because a parent component re-renders frequently and several child components also re-render. How would you investigate and improve it?

- When would `React.memo` help?
- What are `useMemo` and `useCallback` actually solving?
- Why should you avoid adding memoization everywhere?

3 REST API + Express — Request Flow

QUESTION A protected `GET /api/profile` endpoint is returning 401 for a logged-in user. Explain the complete request flow and how you would debug it.

- Where should authentication middleware run?
- How would you distinguish an authentication failure from an authorization failure?
- Which HTTP status codes would you use for 401 and 403?

4 MongoDB + Mongoose — Duplicate Data

QUESTION Your registration API is allowing two users to register with the same email under a race condition. How would you prevent and handle duplicate records?

- Would application-level checking alone be sufficient?
- How can a unique index help?
- How should the API respond when a duplicate-key error occurs?

5 Authentication + Authorization + ImageKit

QUESTION A student is authenticated but should not be allowed to access another student's private resource or upload area. Design the checks you would apply before allowing the request, and explain where ImageKit fits into the upload flow.

- What is the difference between authentication and authorization?
- What information should the server trust from the client?
- How would you prevent a user from changing an ID in the URL and accessing someone else's data?

Future technical topics — As TypeScript, Next.js, advanced React, Redis, deployment, system design or other modules are taught, they will be added to the cumulative interview rotation. Students are expected to revise older topics rather than study only the newest topic.

⌚ Time limit: 2.5 hours

⚙️ Stack: HTML/CSS/JS or React

🚫 Backend not required

Build: Interview Practice Tracker — a responsive dashboard where a student can track their weekly interview-preparation work.

- ▶ **Dashboard** — total questions, completed DSA problems, completed interview questions, machine-coding status.
- ▶ **Question tracker** — add questions with category (DSA / Git / Technical), difficulty, status (Pending / In Progress / Completed).
- ▶ **Search + filters** — filter by category, status and difficulty.
- ▶ **Progress** — visual completion percentage based on stored questions.
- ▶ **Persistence** — LocalStorage so data remains after refresh.
- ▶ **Responsive UI** — desktop + mobile layouts.
- ▶ **States** — empty state, no-search-results state, form validation.
- ▶ **Polish** — clean component structure, readable code, sensible UX.

Deliverables — GitHub repository + live deployed link + 1–2 minute demo video. README should contain setup steps, features and any assumptions.

AI rule — AI/GPT may be used to understand concepts, debug, compare approaches or get hints. The final implementation must be understood by you. You should be able to explain your code in an interview.

Core rule — This is practice, not a competition for who can copy fastest. Use AI/GPT as a learning assistant, not as a replacement for thinking. If you cheat on practice, the only person you cheat is yourself.